

Compte-rendu TP4

```
(kali㉿kali)-[~/R102TP4]
$ sudo python3
[sudo] password for kali:
Python 3.10.8 (main, Nov  4 2022, 09:21:25) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from scapy.all import *
>>> p = Ether() / IP() / ICMP()
>>> p
<Ether type=IPv4 |<IP frag=0 proto=icmp |<ICMP |>>>
>>> █
```

La réponse est un ensemble d'encapsulations des protocoles Ethernet et IP

```
>>> p.summary()
'Ether / IP / ICMP 127.0.0.1 > 127.0.0.1 echo-request 0'
>>> p.show
<bound method Packet.show of <Ether type=IPv4 |<IP frag=0 proto=icmp |<ICMP
|>>>>
```

La commande « p.summary() » affiche l'adresse IP à qui est destinée le message

La commande « p.show » affiche les encapsulations

```
>>> ls(p)
dst      : DestMACField          = 'ff:ff:ff:ff:ff:ff' ('None')
src      : SourceMACField        = '00:00:00:00:00:00' ('None')
type     : XShortEnumField       = 2048                ('36864')
--
version  : BitField (4 bits)     = 4                   ('4')
ihl      : BitField (4 bits)     = None                ('None')
tos      : XByteField            = 0                   ('0')
len      : ShortField            = None                ('None')
id       : ShortField            = 1                   ('1')
flags    : FlagsField           = <Flag 0 ()>        ('<Flag 0 ()>')
)
frag     : BitField (13 bits)    = 0                   ('0')
ttl      : ByteField            = 64                  ('64')
proto    : ByteEnumField         = 1                   ('0')
chksum   : XShortField           = None                ('None')
src      : SourceIPField         = '127.0.0.1'         ('None')
dst      : DestIPField           = '127.0.0.1'         ('None')
options  : PacketListField      = []                  ('[]')
--
type     : ByteEnumField         = 8                   ('8')
code     : MultiEnumField (Depends on 8) = 0                   ('0')
chksum   : XShortField           = None                ('None')
id       : XShortField (Cond)    = 0                   ('0')
seq      : XShortField (Cond)    = 0                   ('0')
ts_ori   : ICMPTimeStampField (Cond) = None                ('33049414')
ts_rx    : ICMPTimeStampField (Cond) = None                ('33049414')
ts_tx    : ICMPTimeStampField (Cond) = None                ('33049414')
```

La commande « ls(p) » affiche les champs des encapsulations des différents protocoles, leurs informations et leurs valeurs

4.

```
>>> ls(Ether)
dst      : DestMACField          = ('None')
src      : SourceMACField        = ('None')
type     : XShortEnumField       = ('36864')

>>> ls(IP)
version  : BitField (4 bits)     = ('4')
ihl      : BitField (4 bits)     = ('None')
tos      : XByteField            = ('0')
len      : ShortField            = ('None')
id       : ShortField            = ('1')
flags    : FlagsField           = ('<Flag 0 ()>')
frag     : BitField (13 bits)    = ('0')
ttl      : ByteField            = ('64')
proto    : ByteEnumField         = ('0')
chksum   : XShortField           = ('None')
src      : SourceIPField         = ('None')
dst      : DestIPField           = ('None')
options  : PacketListField      = ('[]')
```

5.

ETH | IP | ICMP

ETH :

@ MAC de destination: ff:ff:ff:ff:ff:ff

@ MAC source : 00:00:00:00:00:00

IP :

@ IP source : 127.0.0.1

@ IP de destination : 127.0.0.1

6.

ETH | IP | ICMP

ETH :

@ MAC de destination: ff:ff:ff:ff:ff:ff

@ MAC source : 00:00:00:00:00:00

IP :

@ IP source : 127.0.0.1

@ IP de destination : 127.0.0.1

ICMP :

Type : 8(Echo (ping) request)

Code : 0

Checksum : 0xf7ff

7.

```
>>> p = rdpcap("icmp.pcap")
>>> p
<icmp.pcap: TCP:0 UDP:4 ICMP:2 Other:2>
>>> p.show
<bound method _PacketList.show of <icmp.pcap: TCP:0 UDP:4 ICMP:2 Other:2>>
>>>
```

8.

Ethernet

@ MAC dst = ff:ff:ff:ff:ff:ff

@ MAC src = 00:1c:42:70:f5:a5

type(protocole) = ARP

#ARP

hwtype = 0x1

ptype = IPv4

hwlen = 6

plen = 4

op = who-has

hwsrc = 00:1c:42:70:f5:a5

psrc = 10.211.55.4

hwdst = 00:00:00:00:00:00

pdst = 10.211.55.1

9.

```
>>> p[0:3].show()
0000 Ether / ARP who has 10.211.55.1 says 10.211.55.4
0001 Ether / ARP is at 00:1c:42:00:00:18 says 10.211.55.1
0002 Ether / IP / UDP / DNS Qry "b'google.fr.'"
>>>
```

>>>

```
>>> p = Ether('') / IP(dst='192.31.25.1') / ICMP()
>>> p.show()
###[ Ethernet ]###
dst      = 08:00:27:32:34:bf 10.211.55
src      = 08:00:27:b7:44:bd 10.211.55
type     = IPv4 10.211.55
###[ IP ]###
version  = 4
ihl      = None
tos      = 0x0 10.211.55
len      = None
id       = 1
flags    =
frag     = 0 42 bytes on wire (336)
ttl      = 64 1 Src: Parallel70
proto    = icmp 10.211.55
chksum   = None
src      = 192.31.25.14
dst      = 192.31.25.1
\options \
###[ ICMP ]###
type     = echo-request
code     = 0
chksum   = None
id       = 0x0
seq      = 0x0
unused   = ''
```



```
>>> p[IP].dst
192.31.25.14 bytes on wire (src
'192.31.25.1' net 11, src: Parallel 70:
>>> p[IP].src
Resolution Protocol (r
File "<stdin>", line 1
p[IP].src
SyntaxError: invalid syntax
>>> p[IP].src
'192.31.25.14'
>>> p[Ether].dst
'08:00:27:32:34:bf'
>>> p[Ether].src
'08:00:27:b7:44:bd'
>>> p[ICMP].type
8
>>>
```

```
TypeError: Packet.__rdiv__() missing 1 required posi
```

[illegible]

```

File Edit View Help
>>> p = Ether() / IP(dst='192.31.25.1') / ICMP()
>>> rep = sr1(p)
Begin emission:
Finished sending 1 packets.
.*
Received 1 packets, got 1 answers, remaining 0 packets
>>> rep.summary()
'Ether / IP / ICMP 192.31.25.1 > 192.31.25.14 echo-reply 0 / Padding'
>>> rep.show()
###[ Ethernet ]###
  dst      = 08:00:27:b7:44:bd
  src      = 08:00:27:32:34:bf
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 28
  id       = 50330
  flags    = 0
  frag     = 0
  ttl      = 64
  proto    = icmp
  chksum   = 0x3f9
  src      = 192.31.25.1
  dst      = 192.31.25.14
  options  = \
###[ ICMP ]###
  type     = echo-reply
  code     = 0
  chksum   = 0xffff
  id       = 0
  seq      = 0
  unused   = ''
###[ Padding ]###
  load     = '\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'

```

```

>>> p = IP(dst="172.31.25.1") / ICMP()
>>> rep = sr1(p,timeout=5)
Begin emission:
Finished sending 1 packets.
.*
Received 2 packets, got 1 answers, remaining 0 packets

```

Script scan pings

```

File Actions Edit View Help
GNU nano 6.4 scanpings.py
#!/usr/bin/env python3
import argparse
import logging
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
from scapy.all import *
parser = argparse.ArgumentParser(description="Scan ping")
parser.add_argument("max", type=int, help="Valeur du dernier octet max des @IP a scanner")
args = parser.parse_args()
for i in range(1, args.max + 1):
    ip = '192.31.25.' + str(i)
    p = IP(dst=ip) / ICMP()
    conf.verb = 0 # pour ne pas avoir tous les affichages
    intermediaire : mode silencieux ("quiet")
    rep = sr1(p, timeout=5)
    if rep != None:
        print(f"{ip} repond ( host up )")
    else:
        print(f"{ip} ne repond pas ( host down )")

```

```

(kali@kali)-[~/R102TP4]
$ chmod +x scanpings.py

(kali@kali)-[~/R102TP4]
$ sudo ./scanpings.py 55
sudo: unable to execute ./scanpings.py: Permission denied

```

REQUETE DNS

1. ETH | IP | UDP | DNS
- 2.

```
L$ nslookup rt-bethune.univ-artois.fr
;; communications error to 172.18.26.101#53: timed out
Server:      172.18.26.101
Address:     172.18.26.101#53

Non-authoritative answer:
Name:   rt-bethune.univ-artois.fr
Address: 193.49.114.18
```